

net0 Portal Backend

Database Models & API Routes Reference

portal-end/backend | HackGT 13 | September 2026

This document describes the SQLAlchemy models (SQLite by default) and the FastAPI HTTP routes under `portal-end/backend/routes`. Uplink packets from the mesh gateway also write into these tables via `packets/packet_handler.py`. AI enrichment fills `ai_*` fields asynchronously after a report is saved.

1. Entity relationship overview

users (1) --- (*) reports

users (1) --- (*) messages

nodes (standalone; updated by heartbeats)

Reports and messages reference users via `user_id` foreign keys. Deleting a user cascades to their reports and messages.

2. Database models

2.1 users

People who send SOS reports or chat messages over the mesh. Primary key is the mesh `user_id` (not autoincrement).

Fields

user_id (INTEGER PK)

Unique mesh user identifier (1-65535). Comes from the phone/client packet.

name (VARCHAR(32))

Display name from the report packet (may be empty).

phone (VARCHAR(20))

Contact phone from the report packet (may be empty).

first_seen (DATETIME)

UTC timestamp when this user was first created in the portal DB.

last_seen (DATETIME)

UTC timestamp of the most recent packet or API update for this user.

Relationships

- reports - one-to-many Report rows (cascade delete-orphan)
- messages - one-to-many Message rows (cascade delete-orphan)

2.2 reports

SOS / emergency incident records. Created immediately when a report uplink is processed (or via `POST /api/reports`). AI fields start null and are filled later by the AI process pipeline when the LLM is configured.

Fields

id (INTEGER PK)

Autoincrement portal primary key.

msg_id (INTEGER UNIQUE INDEX)

Mesh message ID used for deduplication of retransmits.

attempt (INTEGER)

Retransmit attempt counter from the packet (informational; dedup is on msg_id).

user_id (INTEGER FK INDEX)

Foreign key to users.user_id - the reporter.

origin (INTEGER)

Access-node ID the phone used to enter the mesh.

path (JSON)

List of node IDs hop path toward the gateway, e.g. [1, 2, 9].

category (INTEGER)

Incident category: 0 unknown, 1 medical, 2 trapped, 3 fire, 4 flood, 5 structural, 6 security, 7 hazmat, 8 other.

people (INTEGER)

Number of people involved (0 = unknown).

needs (INTEGER)

Bitflags: injured, rescue, mobility, meds, water, shelter, vulnerable.

gps_lat (FLOAT NULL)

GPS latitude if present (often null until GPS is implemented).

gps_lon (FLOAT NULL)

GPS longitude if present.

gps_accuracy (INTEGER NULL)

GPS accuracy in meters if present.

location (VARCHAR(120))

Human-readable location string from the reporter.

message (VARCHAR(500))

Free-text SOS message body from the reporter.

created_at (DATETIME INDEX)

When the report was stored in the portal.

acked_at (DATETIME NULL)

When an ACK was recorded/sent for this report.

status (VARCHAR(16))

Lifecycle status (default 'received').

ai_priority (INTEGER NULL)

AI-computed dispatch priority 1-5 (null until AI succeeds).

ai_category (INTEGER NULL)

AI-suggested category code (same enum as category); null until AI succeeds.

ai_summary (VARCHAR(1000) NULL)

AI-written operational summary for dispatchers.

ai_responders (JSON NULL)

Responder tags: medical_ems, fire_rescue, law_enforcement, technical_sar, humanitarian_care, coast_guard.

Relationships / constraints

- user - many-to-one User via user_id
- UNIQUE (msg_id) - uq_reports_msg_id

2.3 messages

Chat / reply traffic between responders and users. Uplink user_reply packets create rows with direction='uplink'; portal replies typically use direction='downlink'.

Fields

id (INTEGER PK)

Autoincrement portal primary key.

msg_id (INTEGER NULL UNIQUE INDEX)

Mesh message ID when applicable; used for uplink dedup. May be null for portal-only messages.

direction (VARCHAR(8))

'uplink' (user to portal) or 'downlink' (portal to user).

user_id (INTEGER FK NULL INDEX)

Foreign key to users.user_id when tied to a user; nullable.

reply_to (INTEGER INDEX)

msg_id this message replies to (0 if none).

target_node (INTEGER NULL)

Destination / origin node ID for routing context.

path (JSON NULL)

Optional hop path associated with the message.

sender (VARCHAR(32))

Sender label (often empty on uplink; set for responder downlink).

text (VARCHAR(400))

Message body text.

status (VARCHAR(16))

Delivery/processing status (default 'pending' on create; 'received' for uplinks).

created_at (DATETIME)

When the message row was created.

Relationships / constraints

- user - many-to-one User via user_id (optional)
- UNIQUE (msg_id) - uq_messages_msg_id

2.4 nodes

Mesh radio nodes (relay / access / gateway). Upserted from heartbeat uplinks; also manageable via /api/nodes. Stale nodes are marked offline by a background sweep.

Fields

node_id (INTEGER PK)

Mesh node ID (1-254). Gateway is typically 9.

role (INTEGER)

1 = relay, 2 = access, 3 = gateway.

status (VARCHAR(16) INDEX)

'online' or 'offline' (updated by heartbeats / offline sweep).

clients (INTEGER)

Number of attached clients reported by the node.

path (JSON NULL)

Path info toward gateway as reported in heartbeat.

uptime_s (INTEGER)

Node uptime in seconds.

tx (INTEGER)

Transmit packet/counter statistic from heartbeat.

rx (INTEGER)

Receive packet/counter statistic from heartbeat.

battery (INTEGER NULL)

Battery percentage or level if reported.

neighbors (JSON)

List of neighbor objects (e.g. id, rssi) from heartbeat.

last_seen (DATETIME INDEX)

Last heartbeat / update time; used to mark offline.

3. API routes

Base app also exposes GET / (status) and GET /debug (debug HTML). Interactive OpenAPI docs are at /docs when the server is running. Debug routes are mounted when env DEBUG is not '0' (default on).

3.1 Users - /api/users

GET /api/users

List all users ordered by user_id.

GET /api/users/{user_id}

Get a single user. 404 if missing.

POST /api/users

Create a user.

- Body: user_id (required), name?, phone?
- 409 if user_id already exists. 201 on success.

PATCH /api/users/{user_id}

Update name and/or phone; refreshes last_seen.

- Body (optional): name, phone

DELETE /api/users/{user_id}

Delete user (cascades reports/messages). 204.

3.2 Nodes - /api/nodes

GET /api/nodes

List all nodes ordered by node_id.

GET /api/nodes/{node_id}

Get a single node. 404 if missing.

POST /api/nodes

Create a node.

- Body: node_id (1-254), role?, status?, clients?, path?, uptime_s?, tx?, rx?, battery?, neighbors?
- Default path is [node_id] if omitted. 409 if exists.

PATCH /api/nodes/{node_id}

Partial update; refreshes last_seen.

DELETE /api/nodes/{node_id}

Delete node. 204.

3.3 Reports - /api/reports

GET /api/reports

List reports newest-first.

- Query: user_id?, status?, limit? (1-500, default 100)

GET /api/reports/{report_id}

Get report by primary id, falling back to msg_id lookup.

POST /api/reports

Create a report (also ensures user exists).

- Body: msg_id, user_id, origin, path, category, people, needs, location, message, optional GPS and ai_* fields.
- 409 if msg_id already exists. 201 on success.

PATCH /api/reports/{report_id}

Partial update by id or msg_id.

DELETE /api/reports/{report_id}

Delete report by id or msg_id. 204.

3.4 Messages - /api/messages

GET /api/messages

List messages newest-first.

- Query: user_id?, direction?, reply_to?, limit? (1-500, default 100)

GET /api/messages/{message_id}

Get by primary id, falling back to msg_id.

POST /api/messages

Create a message; auto-creates empty user if user_id given and missing.

- Body: text (required), msg_id?, direction?, user_id?, reply_to?, target_node?, path?, sender?, status?
- 409 if msg_id already exists.

PATCH /api/messages/{message_id}

Partial update by id or msg_id.

DELETE /api/messages/{message_id}

Delete by id or msg_id. 204.

3.5 Stats & health - /api

GET /api/stats

Aggregate counts: users, reports, nodes_total/online/offline, messages, latest_report_at.

GET /api/health

Liveness check. Returns {"status": "online"}.

3.6 AI - /api/ai

POST /api/ai/process

Run AI process on report/packet JSON (LLM currently stubbed).

- Body: category?, people?, needs?, location?, message?, gps_*, msg_id?, prompt_name?, include_prompt?, persist?
- Returns: ai_summary, ai_priority, ai_category, ai_responders, status (stub|ok|parse_error), prompt_name, prompt?
- persist=true requires msg_id and only writes DB when status is ok.

GET /api/ai/prompts

List prompt names stored as *.md under backend/prompts/.

GET /api/ai/prompts/{name}

Load prompt markdown content.

PUT /api/ai/prompts/{name}

Create/overwrite a prompt file.

- Body: { "content": "...markdown with {{placeholders}}..." }

3.7 Debug - /api/debug (DEBUG enabled)

GET /api/debug/export

Export all users, reports, nodes, messages as JSON.

POST /api/debug/import

Import JSON.

- Body: users/reports/nodes/messages arrays and mode=merge|replace.

POST /api/debug/reset

DANGER: truncate all tables.

POST /api/debug/fake/heartbeat

Inject a synthetic heartbeat through packet_handler.

POST /api/debug/fake/report

Inject a synthetic report uplink (stores report; enqueues AI process).

POST /api/debug/fake/user_reply

Inject a synthetic user_reply uplink.

POST /api/debug/seed_random

Insert random demo users/nodes/reports/messages.

4. Notes

- Default DB URL: sqlite:///./portal.db (override with DATABASE_URL).
- Schema migrations live under alembic/versions/; models also drive create_all on startup.
- Packet wire format still includes severity for protocol compatibility, but reports no longer store severity; priority is ai_priority only.
- AI responder values: medical_ems, fire_rescue, law_enforcement, technical_sar, humanitarian_care, coast_guard.
- Live OpenAPI: http://localhost:<port>/docs